

Vulnerability Assessment and Penetration Testing Report



Test Target: Savedate.me

SHIJIMOL S
NOVAVI STUDENT
25-03-2026

Contents

1. Executive Summary	3
1.2 Key Findings	3
1.2.1 Severity	4
1.2.2 Vulnerability Severity Distribution	4
1.2.3 Issue Categorization	5
2. Vulnerability Details	6
2.1 Medium Severity Findings	6
2.1.1 CORS Misconfiguration	6
2.1.2 Absence of Anti-CSRF Tokens	8
2.1.3 WordPress User Enumeration	9
2.2 Low Severity Findings	11
2.2.1 Cookie Without SameSite Attribute	11
2.2.2 Missing / Weak Security Headers	14
2.2.3 Subresource Integrity (SRI) Missing	16
2.2.4 Password Auto-Complete Enabled	17
2.3 Informational Findings	19
2.3.1 Server Information Disclosure	19
2.3.2 Server Information Disclosure via X-Powered-By Header.....	20
2.3.3 Information Disclosure via URL Parameters	22
3. Conclusion	24

1.1 Executive Summary

In March 2026, a Vulnerability Assessment and Penetration Testing (VAPT) was conducted on the web application **SaveDate** (<https://savedate.me>) by Shijimol S as part of an academic evaluation. The objective was to identify security vulnerabilities and assess risks affecting the confidentiality, integrity, and availability of the system.

The assessment involved manual testing, HTTP analysis, and Proof-of-Concept validation. Several security issues were identified, including misconfigurations in CORS, missing CSRF protection, user enumeration, and weak security headers.

These findings indicate areas requiring improvement, and appropriate remediation measures are recommended to enhance the overall security posture of the application.

1.2 Key Findings

The security assessment of the SaveDate web application identified multiple vulnerabilities across different severity levels. The majority of issues are related to improper security configurations, weak access controls, and insufficient protection mechanisms against common web attacks.

The most critical observations include:

- The application is vulnerable to Cross-Origin Resource Sharing (CORS) misconfiguration, allowing requests from untrusted origins with credentials, which may lead to unauthorized data access.
- The absence of anti-CSRF tokens in critical forms enables attackers to perform unauthorized actions on behalf of authenticated users.
- The application is affected by WordPress user enumeration, allowing attackers to identify valid usernames and increasing the risk of brute-force and credential stuffing attacks.
- Several security misconfigurations were identified, including missing or weak security headers and absence of the SameSite attribute in cookies, which weakens protection against client-side attacks.
- External resources are loaded without Subresource Integrity (SRI), increasing the risk of malicious script injection in case of third-party compromise.
- Sensitive user information is exposed through URL parameters, which may lead to data leakage via browser history, logs, and referer headers.
- The application discloses backend technology details (Next.js), which may assist attackers during reconnaissance.

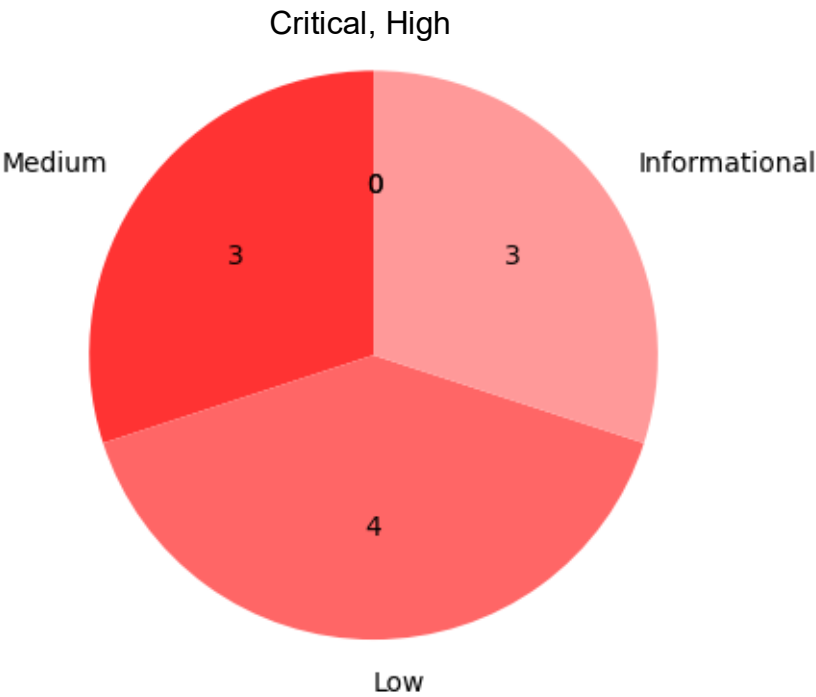
Overall, the findings indicate that the application has moderate security risks, primarily due to misconfigurations and missing security controls rather than complex exploitable vulnerabilities.

1.2.1 Severity

Severity	# of Findings
Critical	0
High	0
Medium	3
Low	4
Informational	3

Table 1: Severity of Vulnerabilities

1.2.2 Vulnerability Severity Distribution



1.2.3 Issue Categorization

Issue Category	# of Findings
Cross-Origin & Request Handling Issues	2
Authentication & Access Control Issues	2
Security Misconfiguration	3
Client-Side Security Issues	2
Information Disclosure	1

Chapter 1

Vulnerability Details

1.1 Medium Findings

1.1.1 Cross-Origin Resource Sharing (CORS) Misconfiguration

Severity: **MEDIUM**

Description

The application is configured with permissive Cross-Origin Resource Sharing (CORS) settings. The server responds with:

```
Access-Control-Allow-Origin: https://hacker.com
Access-Control-Allow-Credentials: true
```

This indicates that the application allows requests from an external, potentially malicious origin (`hacker.com`) while also allowing credentials (cookies, authentication headers) to be included.

Such a configuration is dangerous because it allows untrusted domains to interact with the application and access sensitive data on behalf of authenticated users.

Discovery Method

- Tool Used: Burp Suite (Repeater)
- Intercepted request and modified the `Origin` header.

Steps:

1. Captured a request in Burp Suite
2. Sent it to Repeater
3. Modified header:

```
Origin: https://hacker.com
```

4. Observed server response headers

Proof of Concept (PoC)

Step 1: Modify Origin header

```
Origin: https://hacker.com
```

Step 2: Observe response

Access-Control-Allow-Origin: <https://hacker.com>

Access-Control-Allow-Credentials: true

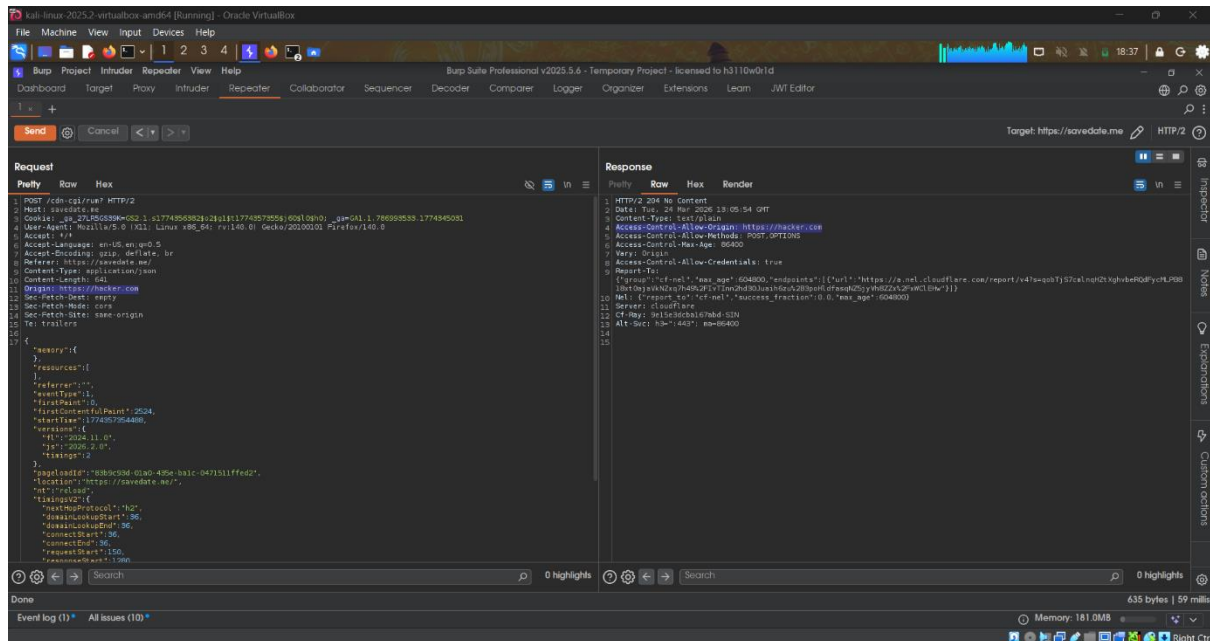


Figure 1.1: Server accepts arbitrary Origin and allows credentials (CORS Misconfiguration)

Impact Analysis

- Unauthorized cross-origin data access
- Session hijacking (if sensitive endpoints exist)
- Data exfiltration
- Increased attack surface for CSRF-like attacks

Especially dangerous when:

- Access-Control-Allow-Credentials: true is enabled
- Sensitive APIs are exposed

Mitigation

- Restrict Allowed Origins
- Do NOT blindly reflect user-supplied Origin headers.
- Disable Credentials if Not Required
- Implement Server-Side Validation
- Use Proper Security Testing

1.1.2 Absence of Anti-CSRF Tokens

Severity: **MEDIUM**

Description

The application contains a form (`/blog/wp-comments-post.php`) that processes user input but does not implement Cross-Site Request Forgery (CSRF) protection.

CSRF tokens are used to ensure that requests are intentionally generated by authenticated users. In this case, the absence of such tokens allows an attacker to craft malicious requests that can be executed in the context of a victim's session.

Discovery Method

- Tool Used: OWASP ZAP
- Manual Verification using browser / proxy (Burp Suite)

Steps:

1. Navigated to blog post page
2. Identified comment submission form
3. Inspected HTML source
4. Observed form structure:

```
<form action="/blog/wp-comments-post.php" method="post">
```

5. Verified absence of:

- Hidden CSRF token field
- Anti-CSRF parameters

Proof of Concept (PoC)

- `https://savedate.me/blog/wp-comments-post.php`
- Observation:
 - POST request does not include any CSRF token
 - Request can be replayed or forged from external domains
- A CSRF attack can be performed by creating a malicious webpage that automatically submits a request:

```
<form action="https://savedate.me/blog/wp-comments-post.php" method="POST">  
  <input type="hidden" name="comment" value="CSRF Test">  
</form>
```

```
<script>
```



```
document.forms[0].submit();  
</script>
```

- When a victim visits this page, the request is sent automatically without their consent.

Impact Analysis

- Unauthorized actions performed on behalf of users
- Spam or malicious content submission
- Potential abuse of authenticated sessions
- Loss of integrity of application data

If user is logged in:

- Actions can be performed without consent
- Session trust is compromised

Mitigation

- Implement CSRF Tokens
- Validate Token Server-Side
- Verify token on every request
- Reject requests without valid token
- Use Framework Protection
- Set SameSite Cookie Attribute
- Check `Origin` / `Referer` headers
- Implement CAPTCHA for sensitive actions

1.1.3 WordPress User Enumeration

Severity: **MEDIUM**

Description

The application is vulnerable to WordPress user enumeration, where valid usernames can be identified through predictable URL parameters. By manipulating the `author` parameter, the application discloses existing usernames via HTTP redirects.

This occurs because the application does not properly restrict access to author archive pages, allowing attackers to enumerate valid user accounts.

Discovery Method

- Manual testing using browser and command-line tool

Steps:

1. Sent a request to:

`https://savedate.me/blog/?author=1`

2. Observed server response headers using:

```
curl -I https://savedate.me/blog/?author=1
```

3. Identified redirect behavior exposing username

Proof of Concept (PoC)

❖ Method 1: Using cURL

Command:

```
curl -I https://savedate.me/blog/?author=1
```

Response:

```
HTTP/2 301
Location: https://savedate.me/blog/author/admin/
```



```
[SHIJI@kali:~]$ curl -I https://savedate.me/blog/?author=1
HTTP/2 301
date: Fri, 27 Mar 2020 13:44:34 GMT
content-type: text/html; charset=UTF-8
location: https://savedate.me/blog/author/admin/
server: cloudflare
x-redirect-by: WordPress
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
strict-transport-security: max-age=31536000; includeSubDomains
cf-cache-status: DYNAMIC
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=031vCaun8C5k2BF1IMw4bIdeK2BUp7sfjpsrLh2B2ZbRHS8UQkmPK2FiFk2Fd4nTbKhk2F0hDhM1ZEM7gsWJT0V0dLeRk2B1wCS
Eh0p0MLAins1k2Z71ToDwASp3iqGmULVnuizYIYASWExriaN3MvX32g3Dk3D"}]}
nel: {"report-to":"cf-nel","success_fraction":0.0,"max_age":604800}
cf-ray: 9e2ed49d3d4e0bbe-SIN
alt-svc: h3=":443"; ma=86400
```

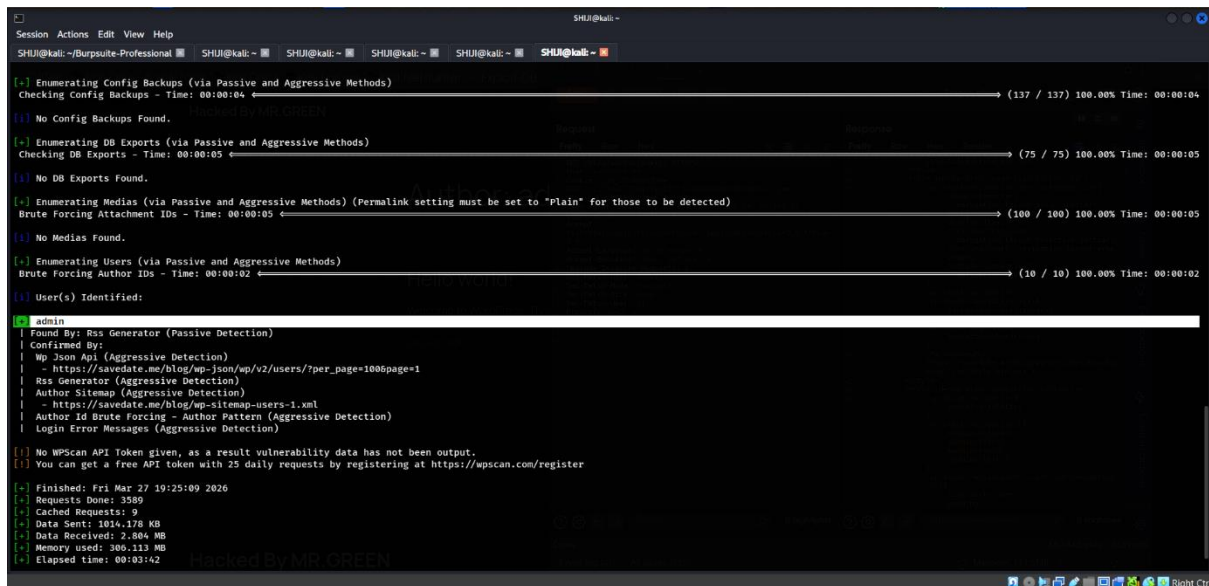
Figure 1.2: cURL shows redirect to /author/admin/ (user enumeration)

❖ Method 2: Using WPScan

Command:

```
wpscan --url https://savedate.me/blog --enumerate u
```

- WPScan successfully enumerates valid usernames
- Confirms user enumeration vulnerability



```
Session Actions Edit View Help
SHUI@kali: ~/Burpsuite-Professional SHUI@kali: SHUI@kali: SHUI@kali: SHUI@kali: SHUI@kali:
[+] Enumerating Config Backups (via Passive and Aggressive Methods)
Checking Config Backups - Time: 00:00:04 → (137 / 137) 100.00% Time: 00:00:04
[+] No Config Backups Found.
[+] Enumerating DB Exports (via Passive and Aggressive Methods)
Checking DB Exports - Time: 00:00:05 → (75 / 75) 100.00% Time: 00:00:05
[+] No DB Exports Found.
[+] Enumerating Medias (via Passive and Aggressive Methods) (Permalink setting must be set to "Plain" for those to be detected)
Brute Forcing Attachment IDs - Time: 00:00:05 → (100 / 100) 100.00% Time: 00:00:05
[+] No Medias Found.
[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:02 → (10 / 10) 100.00% Time: 00:00:02
[+] User(s) Identified:
  admin
  Found By: Rss Generator (Passive Detection)
  Confirmed By:
  Wp Json Api (Aggressive Detection)
    - https://savedate.me/blog/wp-json/wp/v2/users/?per_page=100&page=1
  Rss Generator (Aggressive Detection)
  Author Sitemap (Aggressive Detection)
    - https://savedate.me/blog/wp-sitemap-users-1.xml
  Author Id Brute Forcing - Author Pattern (Aggressive Detection)
  Login Error Messages (Aggressive Detection)
[+] No WPScan API Token given, as a result vulnerability data has not been output.
[+] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register
[+] Finished: Fri Mar 27 19:25:09 2026
[+] Requests Done: 3589
[+] Cached Requests: 9
[+] Data Sent: 1014.178 KB
[+] Data Received: 2.804 MB
[+] Memory used: 306.113 MB
[+] Elapsed time: 00:03:42
Hacked By MR GREEN
```

Figure 1.3: WPScan identifies valid username ‘admin’ (user enumeration)

Impact Analysis

- Attackers can identify valid usernames
- Enables targeted brute-force and credential stuffing attacks
- Increases overall attack surface of the application

Mitigation

- Disable Author Enumeration
- Redirect or Block Author Queries
- Use Security Plugins / Rules
- Implement protections to prevent enumeration attempts
- Enforce Strong Authentication Controls

1.2 Low Findings

1.2.1 Cookie Without SameSite Attribute

Severity: **LOW**

Description

The application sets a cookie (wordpress_test_cookie) without the SameSite attribute. The SameSite attribute is used to control whether cookies are sent along with cross-site requests. In

the absence of this attribute, the browser may include the cookie in cross-origin requests, increasing the risk of Cross-Site Request Forgery (CSRF) attacks.

This misconfiguration may allow attackers to leverage the user's browser to perform unintended actions without their knowledge. Although the affected cookie is non-sensitive, it reflects weak cookie security practices. Proper implementation of the SameSite attribute helps reduce the risk of cross-origin attacks and strengthens overall session security.

Discovery Method

- Manual testing using command-line tool
- Verified HTTP response headers

Steps:

1. Sent request using:

```
curl -I https://savedate.me/blog/wp-login.php
```

2. Observed response headers
3. Identified cookie without SameSite attribute

Proof of Concept (PoC)

- ❖ Method 1: Using Burp Suite

Step 1: Intercept Request

Captured request in Burp Repeater:

```
GET /blog/wp-login.php HTTP/2
Host: savedate.me
```

Step 2: Analyze Response

From response headers:

```
Set-Cookie: wordpress_test_cookie=WP%20Cookie%20check; path=/blog/; Secure;
HttpOnly
```

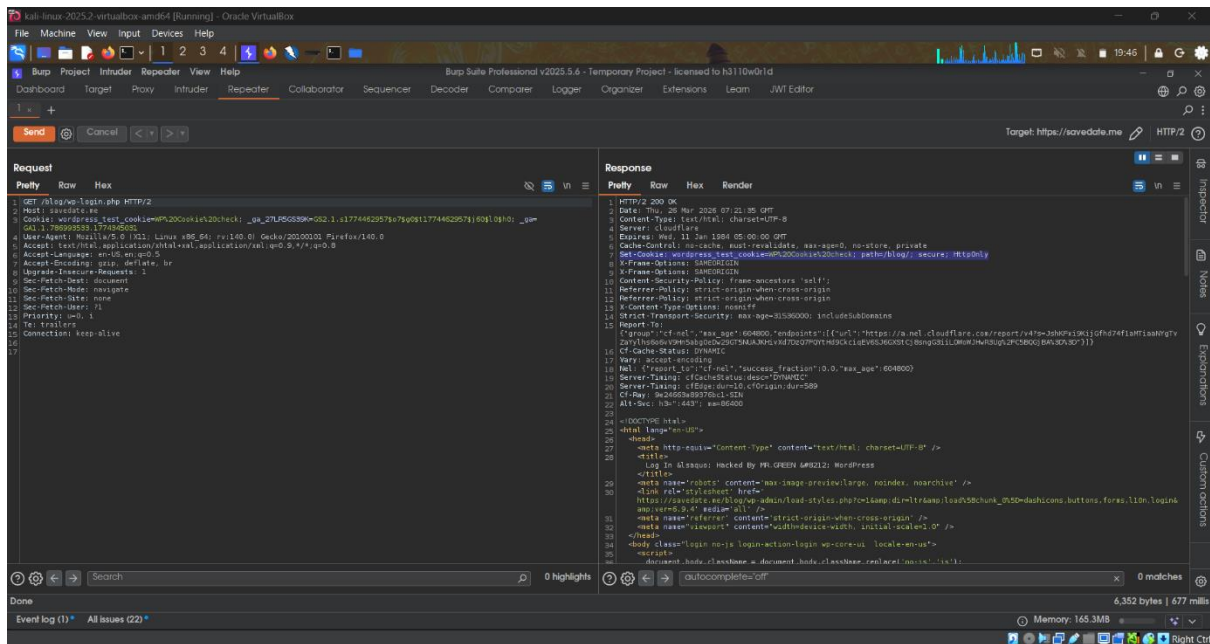


Figure 1.4: Burp Suite showing cookie without SameSite attribute

Method 2: Using cURL

Command:

```
curl -I https://savedate.me/blog/wp-login.php
```

Response:

HTTP/2 200

Set-Cookie: wordpress_test_cookie=WP%20Cookie%20check; path=/blog/; secure; HttpOnly

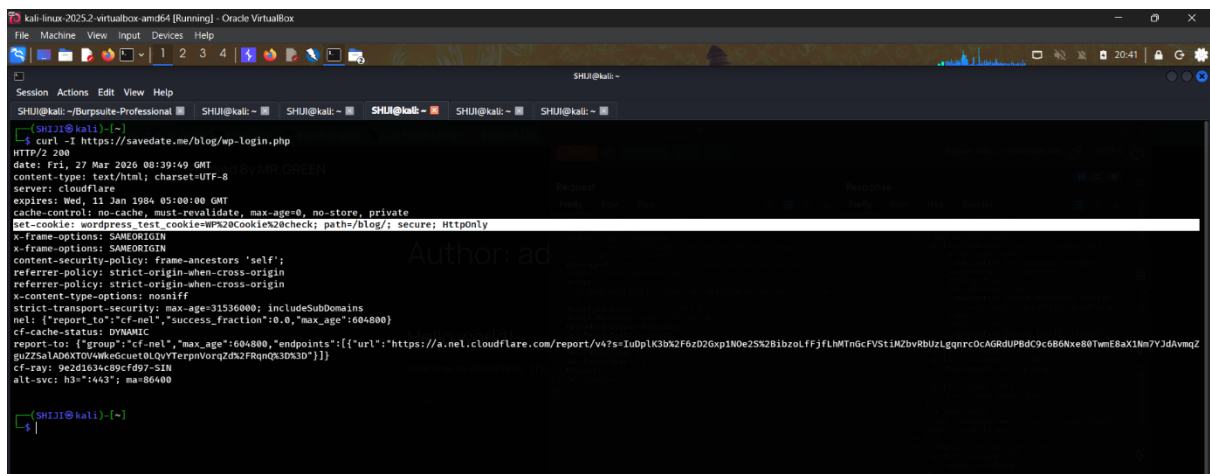


Figure 1.5: cURL output confirming absence of SameSite attribute

Impact Analysis

- Cookies may be sent in cross-site requests
- Increases risk of CSRF attacks

- Weakens session protection mechanisms However, no direct exploitation was observed during testing.

Mitigation

1. Set SameSite Attribute

```
Set-Cookie: wordpress_test_cookie=...; Secure; HttpOnly; SameSite=Lax
```

2. Use Strict Policy for Sensitive Cookies

```
SameSite=Strict
```

3. Apply Secure Defaults

1.2.2 Missing / Weak Security Headers

Severity: LOW

Description

The application does not properly implement essential security headers. Some responses either lack important headers or have weak/misconfigured values.

Specifically:

- Content-Security-Policy (CSP) is weak or improperly configured
- X-Content-Type-Options header is missing in certain responses

Security headers play a critical role in protecting web applications against client-side attacks such as Cross-Site Scripting (XSS) and MIME-type sniffing.

Discovery Method

- Manual testing using Burp Suite and cURL

Steps:

1. Sent requests to multiple endpoints:

```
curl -I https://savedate.me
curl -I https://savedate.me/blog/wp-login.php
```

2. Analyzed response headers
3. Identified missing / weak security headers

Proof of Concept (PoC)

❖ Method 1: Using cURL

Request:

```
curl -I https://savedate.me
```

Response (Evidence):

```
HTTP/2 200 OK
x-powered-by: Next.js

# Missing Security Header:
Content-Security-Policy

# Inconsistent / Missing in some responses:
X-Content-Type-Options
```

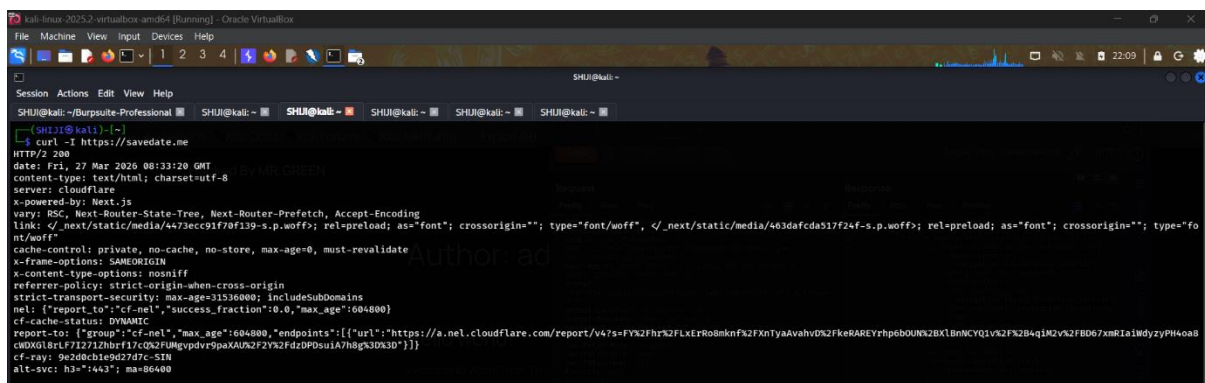


Figure 1.6: Missing CSP in cURL response

❖ Method 2: Using Burp Suite

Steps:

1. Intercept request to:

```
GET / HTTP/2
Host: savedate.me
```

2. Forward request and check Response → Headers

Observed:

- Content-Security-Policy header is not present
- X-Content-Type-Options is missing/inconsistent

Impact Analysis

- Increased risk of Cross-Site Scripting (XSS)
- Allows MIME-type sniffing attacks
- Weakens browser-side protection mechanisms
- May enable injection and content manipulation attacks

Mitigation

- Implement Strong Content Security Policy
- Add X-Content-Type-Options Header
- Apply Security Headers Globally
- Avoid Unsafe Directives

1.2.3 Subresource Integrity (SRI) Missing

Severity: LOW

Description

The application loads external JavaScript resources without implementing the Subresource Integrity (SRI) attribute. SRI ensures that files fetched from third-party sources (such as CDNs) have not been tampered with by verifying their cryptographic hash.

In the absence of the integrity attribute, the browser cannot validate the authenticity of the external scripts, increasing the risk of executing malicious code if the third-party source is compromised.

Discovery Method

The issue was identified through manual testing using Burp Suite and browser inspection. The response HTML was analyzed, and external JavaScript files were found to be included without the `integrity` attribute.

Step 1: Intercept Response (Burp Suite)

- Capture request to the application homepage
- Navigate to Response → HTML

Step 2: Identify External Script

```
<link rel="preload" href="https://www.googletagmanager.com/gtag/js?id=G-XXXXXXX" as="script">
```

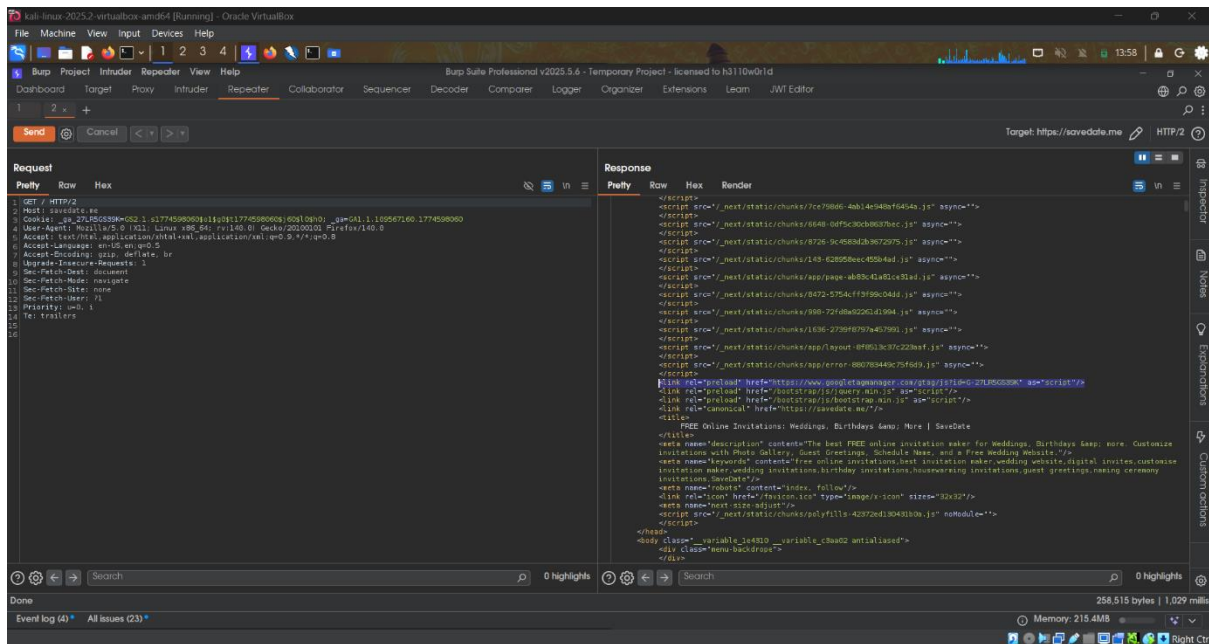



Figure 1.7: External scripts loaded without integrity attribute (SRI missing).

Impact Analysis

- If the external CDN or third-party source is compromised, attackers can inject malicious JavaScript
- Leads to:
 - Cross-Site Scripting (XSS)
 - Session hijacking
 - Data theft or redirection attacks
- Affects all users loading the compromised resource

Mitigation

- Implement Subresource Integrity (SRI) for all external scripts:
- Use trusted and secure CDNs
- Regularly verify third-party dependencies
- Prefer hosting critical scripts locally where possible

1.2.4 Password Auto-Complete Enabled

Severity: **LOW**

Description

The login form allows browsers to store and auto-fill user credentials because the autocomplete attribute is not disabled. By default, browsers may save usernames and passwords and automatically populate them on subsequent visits.

This behavior can expose sensitive credentials, especially when users access the application from shared or public systems.

Discovery Method

The issue was identified through manual testing by inspecting the login form in the browser and Burp Suite. The HTML source of the login page was analyzed, and it was observed that the password input field does not include the attribute `autocomplete="off"`.

Proof of Concept (PoC)

Step 1: Access Login Page

Navigate to:

```
https://savedate.me/blog/wp-login.php
```

Step 2: Inspect Login Form

```
<input type="password" name="pwd">
```

Observation:

```
Missing: autocomplete="off"
```

Browser allows saving and auto-filling credentials

Impact Analysis

- Credentials may be stored in the browser and exposed to other users
- Increased risk on:
 - Shared/public computers
 - Compromised or malware-infected systems
- Attackers with local access can retrieve saved passwords

Mitigation

- Disable autocomplete for sensitive fields:

```
<input type="password" name="pwd" autocomplete="off">
```

- Apply to both username and password fields:

```
<input type="text" name="username" autocomplete="off">  
<input type="password" name="password" autocomplete="off">
```

- Encourage users to avoid saving passwords on shared systems
- Implement multi-factor authentication (MFA) for added security

1.3 Informational Findings

1.3.1 Server Information Disclosure

Severity: INFORMATIONAL

Description

The application discloses underlying server technology through HTTP response headers. Specifically, the header `x-powered-by: Next.js` reveals that the application is built using the Next.js framework.

This information exposure can assist attackers in identifying the technology stack used by the application, enabling them to tailor attacks based on known vulnerabilities associated with that technology.

Discovery Method

The issue was identified through manual testing using cURL and Burp Suite by analyzing HTTP response headers returned by the server.

Proof of Concept (PoC)

Method 1: Using cURL

Request:

```
curl -I https://savedate.me
```

Response (Evidence):

```
HTTP/2 200 OK
x-powered-by: Next.js
```

Method 2: Using Burp Suite

Steps:

1. Intercept request to the application
2. Forward the request
3. Navigate to Response → Headers

Observed:

```
x-powered-by: Next.js
```

Impact Analysis

- Reveals backend technology stack (Next.js)
- Enables attackers to:
 - Identify known vulnerabilities in the framework
 - Perform targeted attacks (framework-specific exploits)
- Assists in reconnaissance phase of an attack

Mitigation

- Remove or disable unnecessary response headers
- Use reverse proxies (e.g., Nginx) to strip headers
- Avoid exposing framework or server details in responses

1.3.2 Server Information Disclosure via X-Powered-By Header

Severity: INFORMATIONAL

Description

The application discloses backend technology information through the `X-Powered-By` HTTP response header. In this case, the header reveals that the application is powered by Next.js.

This information can be leveraged by attackers to identify the underlying technology stack and target known vulnerabilities specific to that framework.

Discovery Method

- Manual testing using Burp Suite and cURL

Steps:

1. Sent request to the application:

```
curl -I https://savedate.me
```

2. Observed response headers
3. Identified technology disclosure

Proof of Concept (PoC)

- ❖ Method 1: Using cURL

Command:

```
curl -I https://savedate.me
```

Response:

```
HTTP/2 200
x-powered-by: Next.js
```

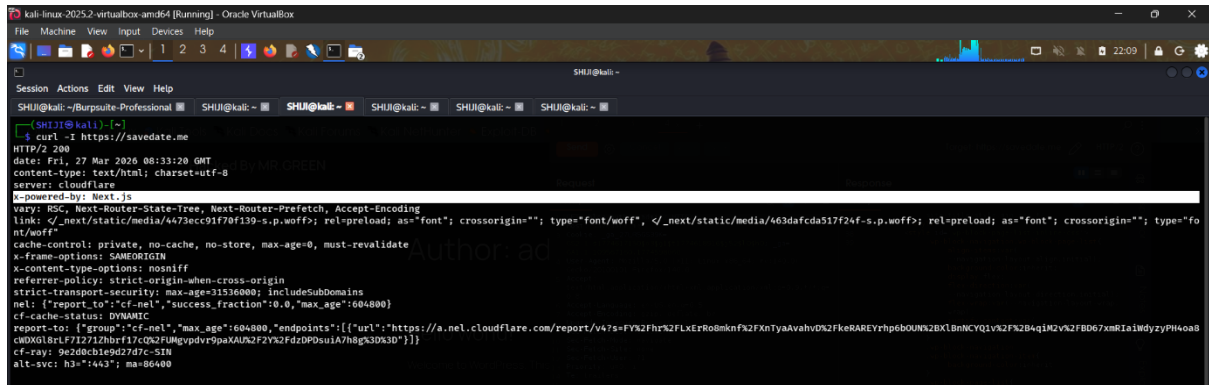


Figure 1.8: cURL output showing ‘X-Powered-By: Next.js’ header

❖ Method 2: Using Burp Suite

Step 1: Intercept Request

```
GET / HTTP/2
Host: savedate.me
```

Step 2: Analyze Response

```
HTTP/2 200 OK
x-powered-by: Next.js
```

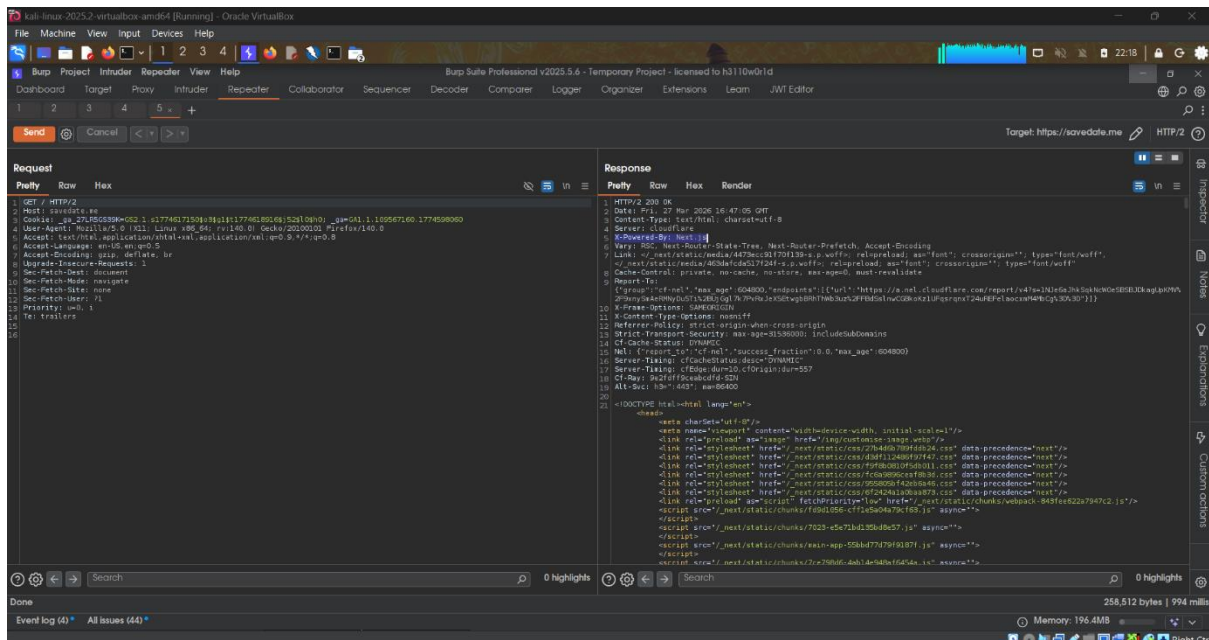


Figure 1.9: Burp Suite response showing ‘X-Powered-By: Next.js’ header”

Impact Analysis

- Reveals internal technology stack
- Helps attackers:
 - Identify known vulnerabilities
 - Plan targeted attacks
- Increases reconnaissance effectiveness

Mitigation

- Disable or remove `X-Powered-By` header from server responses
- Configure Server Securely
- Implement additional security headers to reduce information leakage

1.3.3 Information Disclosure via URL Parameters

Severity: INFORMATIONAL

Description

The application transmits user-supplied data through URL query parameters (e.g., `email`, `phone`, `name`). Sensitive information included in URLs can be exposed to unintended parties because URLs are stored in browser history, server logs, proxy logs, and may be shared via the HTTP Referer header.

This practice increases the risk of information leakage and violates secure data handling principles.

Discovery Method

- Manual testing using browser and proxy tools (e.g., Burp Suite)
- Submitted data through input forms and observed that parameters were appended to the URL

Steps:

1. Entered sample data into form fields (name, email, phone)
2. Submitted the form
3. Observed URL in browser address bar / intercepted request

Impact Analysis

- Sensitive data stored in:
 - Browser history

- Server and proxy logs
 - Shared links
- Data leakage via:
 - Referrer headers to third-party domains
 - Shoulder surfing / shared systems
- Privacy risks and potential compliance issues

Chapter 2

Conclusion

The Vulnerability Assessment and Penetration Testing (VAPT) conducted on the SaveDate web application identified a range of security issues across Medium, Low, and Informational severity levels. While no Critical or High-risk vulnerabilities were discovered, the presence of multiple Medium severity findings indicates that the application is exposed to realistic and exploitable attack scenarios.

Key issues such as CORS misconfiguration, absence of CSRF protection, and user enumeration pose significant risks, as they may allow unauthorized access, execution of unintended actions, and exposure of valid user accounts. These vulnerabilities directly impact the application's security controls related to authentication, request validation, and cross-origin communication.

In addition, several Low severity vulnerabilities, including missing security headers, lack of SameSite cookie attributes, and absence of Subresource Integrity (SRI), highlight weaknesses in secure configuration and client-side protection mechanisms. Although individually these issues may have limited impact, collectively they increase the overall attack surface and make the application more susceptible to exploitation.

The assessment also identified informational findings, such as technology disclosure and exposure of sensitive data in URL parameters. While these do not directly compromise the system, they provide valuable information to attackers during the reconnaissance phase and may contribute to more targeted attacks.

Overall, the application demonstrates moderate security risk, primarily due to misconfigurations and missing security controls rather than complex vulnerabilities. Addressing these issues through proper implementation of security best practices—such as enforcing strict access controls, validating user requests, securing cookies, and applying robust security headers—will significantly enhance the application's resilience.

It is recommended that remediation efforts prioritize Medium severity vulnerabilities, followed by Low and Informational findings, to ensure a structured and effective improvement in the application's security posture.